

OpenZero 1.0: A Local-First Agent Runtime with Observable Serving-Path Boundaries

Shafaet Brady Hussain Independent researcher and developer, United Kingdom 18 August 2026

Abstract

This paper consolidates OpenZero as a local-first agent runtime rather than a claim of a universally superior model. The evaluated snapshot combines a browser interface, an Ollama-compatible local inference lane, optional cloud fallback, model inventory and repair controls, structured operator tools, raw-shell fallback, retrieval/memory components, integrity manifests, optional voice, watchdog utilities, and an offline distribution path. I define local-first as a routing and deployment property: selected inference and control functions can operate on the user's node, while network tools, cloud models, remote administration, and software updates remain explicitly non-local.

Six principal Python modules compiled successfully in a frozen source check on 18 August 2026. Historical ten-item route benchmarks show the practical importance of wrapper boundaries: eleven tested routes ranged from 0/10 to 9/10 exact answers, and several HTTP-successful paths emitted control text instead of task answers. These observations diagnose system integration; they do not establish general intelligence, model superiority, privacy for every mode, offline completeness, safe autonomous tool use, or production security. Source inspection also identifies a high-risk authority boundary: file mutation, removal, remote SSH/SCP, URL retrieval, and shell execution require stronger policy mediation than prompt instructions alone.

Keywords: local-first AI, agent runtime, Ollama, serving path, tool authority, offline deployment, reproducibility

1. Research question and contribution

The research question is: **What can be claimed and tested about a local-first agent runtime when model, wrapper, tool, network, and operator boundaries are reported separately?**

OpenZero's contribution is an integration architecture built around inspectable local services. It is not a new foundation-model architecture. This paper contributes:

1. a formal component and authority model for the runtime;
2. a definition of local-first that permits declared optional network dependencies;
3. a frozen source ledger and syntax validation;
4. a consolidation of dated route benchmarks and failure evidence;

5. a threat model for agent tools, files, networks, integrity state, and model output; and
6. executable release-boundary checks in a public companion.

2. Evidence boundary

The primary source snapshot is the local DEPLOY_OPENZERO corpus plus already public benchmark artifacts in the ResearchForumOnline repository. I did not start the runtime, change its integrity state, invoke its doctor, install software, call cloud providers, contact remote hosts, or run agent tools. This avoids mutating an operational corpus and prevents unreviewed external actions.

Six files were compiled with Python's `py_compile`: `zero_core.py`, `openzero_doctor.py`, `openzero_watchdog.py`, `brain/app.py`, `brain/integrity.py`, and `brain/openzero_config.py`. Compilation passed for all six. This proves parseability under the current Python interpreter, not runtime correctness.

Principal snapshot hashes include:

Artifact	SHA-256
<code>zero_core.py</code>	<code>fc28d9e8b57adab88ff1babaf968f9bedb337db4f013216d2bae2ed2dc2f6dbb</code>
<code>openzero_doctor.py</code>	<code>8295f3757da669fa041ceb11de6bf22622440db0b0e73bd09fdc35966eb13ff2</code>
<code>brain/app.py</code>	<code>44acb65cf78dd91506fe5298a7a94bb9e3a6d9b9230b85571090c8c5ae94250d</code>
<code>brain/integrity.py</code>	<code>0467ae64820138172f6f50ba1ea455e5f70d1d55bec30bdf0208025157ee395c</code>
offline builder	<code>93ea9b8ab220f74d00203d4f4cde35795cedd7f27e2784b2e5e74852f9ef5986</code>
public route summary	<code>a36e99a0812481d651cc39789b353b086485ba8ac3fcfbbe8818583c9d95f2c5</code>

3. Runtime model

Let the runtime be

$OZ = (U, C, M, G, T, V, S, D, L)$

where *U* is the interface, *C* configuration, *M* model selection, *G* generation route, *T* tool authority, *V* verification/integrity state, *S* storage and memory, *D* deployment/repair, and *L* logs and observable state.

3.1 Interface and configuration

The Flask-based application exposes status, configuration, model inventory, local-model installation/removal, Ollama status/repair, BitNet add-on controls, integrity status, and agent interaction routes. Configuration can change runtime behaviour, so a reproducible record must bind effective configuration, not only source hashes.

3.2 Model and generation lanes

The local lane calls Ollama on loopback (`127.0.0.1:11434`) and supports model selection/normalisation. An optional BitNet lane is treated as a separate runtime, with fallback to Ollama. Optional Groq/cloud routing exists. Therefore local-first does not mean network-free: the active lane and every fallback must be logged.

3.3 Tools

The runtime advertises structured file, archive, web, SSH, and SCP operations, plus raw shell and browser/OSINT/voice channels. Structured tools improve typing and logging but are not automatically safe. Read, write, append, replace, remove, archive extraction, network fetch, remote command, and shell operations cross distinct authority boundaries.

3.4 Integrity and recovery

The integrity module maintains a manifest and a sealed ethics-policy state using local key material. It can detect or restore selected files. This is an application integrity mechanism, not secure boot, hardware attestation, or protection from an attacker who controls the same account, key, or process. The doctor creates directories, fills defaults, changes permissions, and rebuilds integrity state; it is deliberately mutating and was not run for this paper.

3.5 Offline distribution

The offline builder is designed to package source, Python wheels, Node dependencies/runtime, PM2, an Ollama binary, local model store, and optional voice wheels. Documentation estimates a 15-25 GB bundle after model inclusion. An offline target still needs a compatible Linux base and libraries. The paper did not build or install this bundle, so “air-gap ready” remains a documented design claim rather than a verified result here.

4. Local-first definition

A runtime is **local-first** when:

- local inference is a first-class selectable path;
- local state and model identity are visible to the operator;
- cloud use is optional and declared at action time;
- loss of cloud connectivity has a defined local or fail-closed outcome;
- tools disclose when they contact a network or remote host; and
- logs distinguish local generation, cloud generation, tool output, fallback, and failure.

Local-first does not imply local-only, private-by-default for every feature, secure autonomy, low cost on every machine, or equivalent model quality. Web search, URL fetch, SSH, SCP, cloud inference, model download, package installation, and update checks are network operations.

5. Serving-path evaluation

5.1 Historical protocol

On 8 July 2026, eleven named local routes were evaluated through the OpenZero/Ollama-compatible path on ten objective prompts, temperature zero, and a 64-token cap. The result is a diagnostic smoke test, not a general model ranking.

Route	Exact / 10	Mean latency (s)
spectra8-q8:latest	9	15.035
gemma3:12b	8	5.748
hermes3:8b-llama3.1-q5_K_M	8	7.505
talktoaizero-q6:latest	7	4.096

Route	Exact / 10	Mean latency (s)
qwen2.5:1.5b	7	1.236
qwen2.5:3b	7	1.617
glm4:9b-q5	6	9.966
spectramind3-q8:latest	1	4.945
microspectramind-q8:latest	0	4.993
spectramindz-q8:latest	0	24.830
talktoaiq-f16:latest	0	43.258

Several failing routes returned HTTP success while echoing OpenZero control text. This demonstrates that transport success, model loading, wrapper correctness, and task correctness are different variables.

5.2 Direct versus wrapped evidence

A separate dated Spectra8 direct-route snapshot achieved 9/10 while the authenticated OpenZero wrapped route later achieved 3/10. The comparison is not perfectly controlled, but it is evidence against attributing every wrapped failure to the model artifact. A rigorous evaluation must freeze weights, quantisation, chat template, prompt, context, runtime, wrapper, sampling, hardware, and scorer.

5.3 Interpretation

The benchmark supports a narrow claim: OpenZero can route multiple local models, and serving-path contamination is observable. It does not show that OpenZero improves answer quality, that local models beat hosted providers, or that a 9/10 route is broadly capable. It also reveals why visible raw outputs and failures are essential.

6. Agent authority and safety

The most consequential runtime boundary is not the model score but tool authority. The evaluated source can perform file writes/removal, archive extraction, URL retrieval, SSH/SCP, and raw shell execution. Model-generated tags are parsed into actions. Prompt instructions such as “prefer structured tools” are not an adequate security boundary.

A safer profile requires:

1. an operator-scoped workspace root with path canonicalisation;
2. separate allowlists for read, write, delete, network, remote, and shell classes;
3. action-time confirmation for destructive, remote, credential, privilege, or persistence changes;
4. time, size, output, recursion, host, and command limits;
5. no implicit credential discovery or transmission;
6. immutable audit records containing requested, authorised, executed, and observed states;
7. cancellation and emergency stop checked between tool actions; and
8. denial when model output is malformed, unknown, or attempts to expand authority.

The current source contains useful state/cancellation and structured-action machinery, but this paper does not certify the complete enforcement of these controls.

7. Threat model

Threats include prompt injection in local documents or fetched pages; malicious model output; unsafe archive paths; path traversal; command injection; unintended remote access; sensitive logs; exposed Flask endpoints; weak configuration; dependency/model substitution; integrity-key compromise; and fallback from local to cloud without clear consent.

Assets include local files, credentials, model weights, configuration, prompt/memory history, remote hosts, network identity, and operator trust. The model is untrusted input to the policy layer. Tool output is untrusted input to subsequent model turns.

The integrity manifest can detect selected file changes, but it cannot establish a trusted root if the process, key, manifest, or operating account is compromised. Encryption or hashing of local state does not solve authorisation.

8. Reproducible conformance

OpenZero Local Runtime conformant requires a declared local inference endpoint, active model identity, effective prompt/template hash, generation parameters, and explicit failure state. **OpenZero Local-First conformant** additionally requires declared network transitions and offline fallback behaviour. **OpenZero Agent conformant** additionally requires typed tool schemas, policy decisions, bounded execution, operator-visible logs, cancellation, and fail-closed unknown actions.

The release companion checks a narrow machine-readable boundary matrix: every action class must declare locality, mutability, and confirmation policy. Passing does not prove that the production code enforces the matrix.

9. Falsifiable evaluation programme

Future tests should compare direct model, Ollama chat template, OpenZero wrapper without tools, and full agent route on the same frozen tasks. Primary outcomes should include exact task result, template leakage, unsupported tool calls, correct abstention, tool-policy violations, latency, memory, energy, and recovery after cancellation.

The local-first claim is falsified if a nominally local action contacts an undeclared network service. The fail-closed claim is falsified if an unknown or malformed tool executes. The wrapper-neutrality claim is falsified when the wrapper materially degrades results under controlled inputs. The offline claim is falsified if a documented offline install requires undeclared downloads.

10. Claim ledger

Claim	Status
The snapshot contains local Ollama routing and optional separate/cloud lanes.	Supported by source inspection.
Six principal Python modules parse under the current interpreter.	Supported by 6/6 compilation check.
OpenZero works fully offline on every Linux machine.	Not established.
HTTP 200 proves useful inference.	Rejected by benchmark failures.
The best 10-item score proves general model superiority.	Rejected.
Structured tool tags alone make autonomous actions safe.	Rejected.
Integrity manifests provide hardware-rooted attestation.	Rejected.
Local-first guarantees privacy for network tools and cloud fallback.	Rejected.

11. Limitations

This is an author-led source audit and historical benchmark consolidation. I did not execute the application, its mutating doctor, tools, model downloads, cloud routes, voice stack, watchdog, installer, or offline bundle. Syntax compilation is weak evidence. Benchmark tasks are too small for capability ranking and come from dated runtime snapshots.

The source exposes powerful operational features whose safety needs a separate adversarial evaluation. No penetration test, sandbox escape test, multi-user authorisation study, privacy audit, energy measurement, accessibility study, or independent review is reported.

12. Conclusion

OpenZero is best understood as a local-first integration runtime with multiple inference and tool boundaries. Its strongest evidenced result is methodological: wrapper behaviour can convert a loaded, HTTP-successful local route into a failed task, and raw outputs are necessary to diagnose that failure.

Local control is valuable only when authority is equally explicit. The next milestone is a frozen conformance harness that measures model, wrapper, and tool policy separately, followed by an independently reproducible offline installation and adversarial tool-boundary evaluation.

Data and code availability

The paper, public route results, action-boundary matrix, verifier, and hashes are released in the ResearchForumOnline research repository. Credentials, private evaluations, model weights, operational hosts, and user data are excluded.

Conflict of interest and AI-use disclosure

I created and maintain OpenZero-related components. This is not an independent audit. AI-assisted tools supported corpus search, source reading, test orchestration, risk analysis, and editorial refinement. I am responsible for the released claims and limitations.

References

1. Sculley, D., et al. Hidden Technical Debt in Machine Learning Systems. *NeurIPS* 28, 2015.
2. Mitchell, M., et al. Model Cards for Model Reporting. *FAT* 2019. <https://doi.org/10.1145/3287560.3287596>
3. Liang, P., et al. Holistic Evaluation of Language Models. *TMLR*, 2023. <https://doi.org/10.48550/arXiv.2211.09110>
4. NIST. *Artificial Intelligence Risk Management Framework 1.0*. 2023. <https://doi.org/10.6028/NIST.AI.100-1>
5. OWASP Foundation. *Top 10 for Large Language Model Applications*. <https://genai.owasp.org/>
6. Ollama. *API documentation*. <https://github.com/ollama/ollama/blob/main/docs/api.md>

Appendix. Supersession statement

This paper supersedes the generic working paper *OpenZero and Local-First AI Nodes for Low-Cost Sovereign Agents* for runtime claims. It preserves the local-control goal but replaces broad “sovereign agent” language with observable lanes, tool authority, failure states, and falsifiable tests.